

Algorithmic Analysis of Stochastic Gradient Descent for Malaria Cell Classification Using a Convolutional Neural Network

Felix Rolando Gonzalez Zapatanga

Yachay Tech University
Analysis of Algorithms

Presentation II

Selected algorithm: **Stochastic Gradient Descent (SGD)**

Project Focus and Motivation

Main idea

This project studies **SGD as an optimization algorithm**, not just a neural network application.

- Application: binary malaria cell classification.
- Classes: **Parasitized** and **Uninfected**.
- Optimizer: pure mini-batch SGD.
- Adam, transfer learning, and ensembles were removed to isolate SGD behavior.

Analysis goals

- Formal definition.
- Computational complexity.
- Convergence behavior.
- Runtime vs. quality trade-off.

Binary classification

$$y = \begin{cases} 1, & \text{parasitized cell,} \\ 0, & \text{uninfected cell.} \end{cases}$$

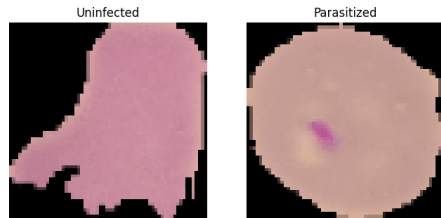
$$f_{\theta}(x_i) \rightarrow [0, 1]$$

$$\hat{y} = 1 \text{ if } f_{\theta}(x_i) \geq 0.5$$

- Input: RGB cell image.
- Output: probability of infection.
- The model learns parameters θ by minimizing binary cross-entropy.
- The result is for algorithmic analysis, not medical diagnosis.

Dataset and Preprocessing

- Total images: **1067**.
- Uninfected: **567**.
- Parasitized: **500**.
- Train/test split: **80% / 20%**.
- Training images: **853**.
- Test images: **214**.



Preprocessing

Resize to $64 \times 64 \times 3$ and normalize pixels to $[0, 1]$.

Formal Definition of Mini-Batch SGD

Objective function

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N L(f_{\theta}(x_i), y_i)$$

Binary cross-entropy

$$L(\hat{y}, y) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

SGD update rule

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(\theta_t; B_t)$$

η is the learning rate and B_t is the mini-batch used at iteration t .

Mini-batch SGD procedure

- ➊ Initialize CNN parameters θ .
- ➋ Shuffle the training set.
- ➌ Split data into mini-batches.
- ➍ For each mini-batch:
 - forward propagation,
 - compute loss,
 - backpropagation,
 - update parameters.
- ➎ Repeat for 30 epochs.

Design paradigm

- Iterative optimization.
- Approximate gradient using mini-batches.
- Low-cost updates.
- Sensitive to learning rate and batch size.

$$\theta \leftarrow \theta - \eta \nabla L$$

CNN Architecture and Implementation

Layer	Output shape	Parameters
Conv2D, 16 filters	$62 \times 62 \times 16$	448
MaxPooling2D	$31 \times 31 \times 16$	0
Conv2D, 32 filters	$29 \times 29 \times 32$	4,640
MaxPooling2D	$14 \times 14 \times 32$	0
Flatten	6272	0
Dense, 64 neurons	64	401,472
Dense, 1 neuron	1	65
Total	—	406,625

- Python with TensorFlow/Keras.
- Optimizer: **SGD only**.
- Loss: binary cross-entropy.
- Output activation: sigmoid.
- Goal: clear algorithmic analysis.

Updates per epoch

$$U = \left\lceil \frac{N}{B} \right\rceil$$

$$U_{B=32} = \left\lceil \frac{853}{32} \right\rceil = 27$$

$$U_{B=128} = \left\lceil \frac{853}{128} \right\rceil = 7$$

Training cost

$$T_{epoch} = O(N(C_f + C_b))$$

$$T_{total} = O(E \cdot N \cdot (C_f + C_b))$$

With fixed architecture: $T_{total} = O(E \cdot N)$.

Best, average, and worst cases differ mainly by the number of epochs needed to reach acceptable convergence.

Experimental Setup

Fixed conditions

- Same CNN architecture.
- Same train/test split.
- Same preprocessing pipeline.
- Training length: **30 epochs**.

Compared SGD configurations

- $\eta = 0.01$, batch size 32.
- $\eta = 0.001$, batch size 32.
- $\eta = 0.01$, batch size 128.

Evaluation metrics

Accuracy, precision, recall, F1-score, confusion matrix, convergence curves, and training time.

Empirical Results After 30 Epochs

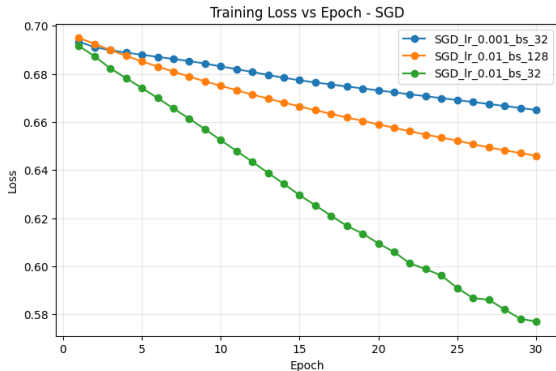
Experiment	Accuracy	Precision	Recall	F1	Time (s)
SGD lr=0.01, bs=32	0.7196	0.6515	0.8600	0.7414	78.63
SGD lr=0.01, bs=128	0.6028	0.7143	0.2500	0.3704	73.52
SGD lr=0.001, bs=32	0.5841	0.7200	0.1800	0.2880	81.68

Main result

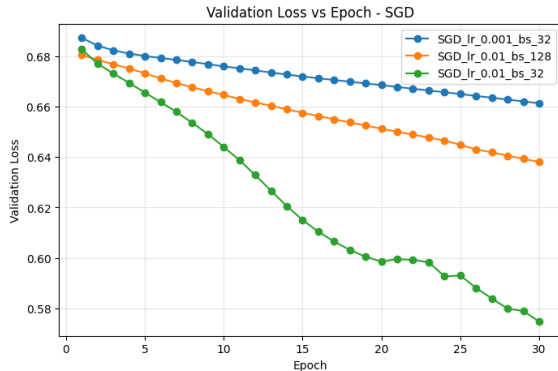
The best overall configuration was **learning rate 0.01 with batch size 32**. It achieved the highest accuracy, recall, and F1-score.

Recall is especially important here because false negatives correspond to infected cells classified as uninfected.

Convergence Behavior: Loss Curves



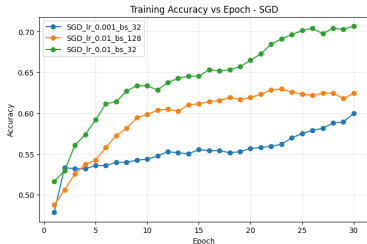
Training loss



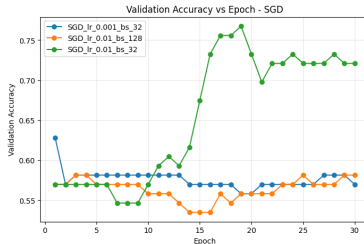
Validation loss

- SGD with $\eta = 0.01$ and batch size 32 reduced loss more effectively.
- The $\eta = 0.001$ configuration converged more slowly.

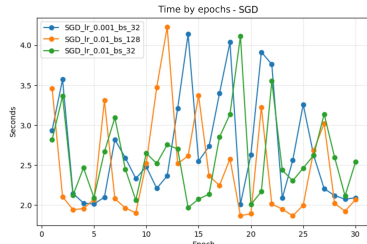
Accuracy and Runtime Trade-off



Training accuracy



Validation accuracy



Time by epoch

Batch size 128 was slightly faster per epoch, but it produced weaker recall and F1-score. The best practical trade-off was batch size 32 with learning rate 0.01.

Best Model: Confusion Matrix, New Prediction, and Conclusions

Confusion Matrix - Best SGD Model

True	Uninfected	60	54
	Parasitized	10	90
		Uninfected	Parasitized
		Predicted	

Best SGD confusion matrix

Prediction: Uninfected
Probability: 0.3940



New image: Uninfected, $P = 0.3940$

Conclusions

- SGD is simple and computationally efficient per update.
- Performance depends strongly on learning rate, batch size, and epochs.
- Best result: 71.96% accuracy and 86.00% recall.
- The model is educational, not a medical diagnostic system.

Questions?